Python

```python
import json

def format_ipynb_for_word(file_path):
    with open(file_path, 'r', encoding='utf-8') as f:
        nb = json.load(f)

    output = []
    for cell in nb.get('cells', []):
        cell_type = cell.get('cell_type')
        source = "".join(cell.get('source', []))

        if cell_type == 'markdown':
            output.append(f"{source}\n")

        elif cell_type == 'code':
            if not source.strip():
                continue
            output.append(f"**Code Block:**\n```
```text?code_stdout&code_event_index=1
**Code Block:**
```python
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
```
```

---

Find some relation between features and loan status

**Code Block:**

Python

```python
data = pd.read_csv('loan.csv')
data.shape
```

**[Output]**

Plaintext

```
(614, 13)
```

---

**Code Block:**
```

```python
data.head()
```

**[Output]**

```Plaintext
   Loan_ID Gender Married Dependents     Education Self_Employed  \
0  LP001002   Male      No          0      Graduate            No
1  LP001003   Male     Yes          1      Graduate            No
2  LP001005   Male     Yes          0      Graduate           Yes
3  LP001006   Male     Yes          0  Not Graduate            No
4  LP001008   Male      No          0      Graduate            No

   ApplicantIncome  CoapplicantIncome  LoanAmount  Loan_Amount_Term  \
0             5849                0.0         NaN             360.0
1             4583             1508.0       128.0             360.0
2             3000                0.0        66.0             360.0
3             2583             2358.0       120.0             360.0
4             6000                0.0       141.0             360.0

   Credit_History Property_Area Loan_Status
0             1.0         Urban           Y
1             1.0         Rural           N
2             1.0         Urban           Y
3             1.0         Urban           Y
4             1.0         Urban           Y  ```
```

(Table/Dataframe result)

---

**Code Block:**

```python
data.dtypes
```

**[Output]**

```Plaintext
Loan_ID              object
Gender               object
Married              object
Dependents           object
Education            object
Self_Employed        object
```

ApplicantIncome int64
CoapplicantIncome float64
LoanAmount float64
Loan_Amount_Term float64
Credit_History float64
Property_Area object
Loan_Status object
dtype: object```

---

**Code Block:**
```python
data["Dependents"].value_counts()
```

**[Output]**

Plaintext

```
Dependents
0    345
1    102
2    101
3+    51
Name: count, dtype: int64```
```

---

**Code Block:**
```python
data = data.drop('Loan_ID',axis = 1)
```

---

**Code Block:**

Python

```
data.head()
```

**[Output]**

Plaintext

```
 Gender Married Dependents     Education Self_Employed  ApplicantIncome  \
0   Male      No          0      Graduate            No             5849
```

```
1   Male     Yes          1      Graduate           No            4583
2   Male     Yes          0      Graduate          Yes            3000
3   Male     Yes          0  Not Graduate           No            2583
4   Male      No          0      Graduate           No            6000

   CoapplicantIncome  LoanAmount  Loan_Amount_Term  Credit_History  \
0                0.0         NaN             360.0             1.0
1             1508.0       128.0             360.0             1.0
2                0.0        66.0             360.0             1.0
3             2358.0       120.0             360.0             1.0
4                0.0       141.0             360.0             1.0

  Property_Area Loan_Status
0         Urban           Y
1         Rural           N
2         Urban           Y
3         Urban           Y
4         Urban           Y  ```
```

(Table/Dataframe result)

---

**Code Block:**

```python
data.describe()
## shows only numerical data
```

**[Output]**

Plaintext

```
      ApplicantIncome  CoapplicantIncome  LoanAmount  Loan_Amount_Term  \
count       614.000000         614.000000  592.000000         600.00000
mean       5403.459283        1621.245798  146.412162         342.00000
std        6109.041673        2926.248369   85.587325          65.12041
min         150.000000           0.000000    9.000000          12.00000
25%        2877.500000           0.000000  100.000000         360.00000
50%        3812.500000        1188.500000  128.000000         360.00000
75%        5795.000000        2297.250000  168.000000         360.00000
max       81000.000000       41667.000000  700.000000         480.00000

       Credit_History
count      564.000000
mean         0.842199
std          0.364878
min          0.000000
```

25%  1.000000
50%  1.000000
75%  1.000000
max  1.000000  ```
(Table/Dataframe result)

---

**Code Block:**
```python
data.describe(include = ['object'])
```

**[Output]**

```Plaintext
      Gender Married Dependents Education Self_Employed Property_Area  \
count     601     611        599       614           582           614
unique      2       2          4         2             2             3
top      Male     Yes          0  Graduate            No     Semiurban
freq      489     398        345       480           500           233

       Loan_Status
count          614
unique           2
top              Y
freq           422  ```
(Table/Dataframe result)

---

**Code Block:**
```python
#missing values
data.isna().sum()
```

**[Output]**

```Plaintext
Gender              13
Married              3
Dependents          15
Education            0
Self_Employed       32
```
```

ApplicantIncome 0
CoapplicantIncome 0
LoanAmount 22
Loan_Amount_Term 14
Credit_History 50
Property_Area 0
Loan_Status 0
dtype: int64```

---

**Code Block:**
```python
cat_cols = data.dtypes =='object'
cat_cols = list(cat_cols[cat_cols].index)
cat_cols
```

**[Output]**

Plaintext

```
['Gender',
 'Married',
 'Dependents',
 'Education',
 'Self_Employed',
 'Property_Area',
 'Loan_Status']```
```

---

**Code Block:**
```python
num_cols = data.dtypes !='object'
num_cols = list(num_cols[num_cols].index)
num_cols
```

**[Output]**

Plaintext

```
['ApplicantIncome',
 'CoapplicantIncome',
 'LoanAmount',
 'Loan_Amount_Term',
```

'Credit_History']```

---

**Code Block:**
```python
cat_cols.remove('Loan_Status') # Loan_Status is target, so remove from features
cat_cols
```

**[Output]**

Plaintext

```
['Gender',
 'Married',
 'Dependents',
 'Education',
 'Self_Employed',
 'Property_Area']```
```

---

# Univariate Analysis

**Code Block:**
```python
data['Loan_Status'].value_counts()
target = 'Loan_Status'
```

---

**Code Block:**

Python

```
sns.countplot(data=data, x='Loan_Status')
plt.show()
```

**[Output]**

Plaintext

```
<Figure size 640x480 with 1 Axes>```
```
(Visualization/Plot)

---

**Code Block:**
```python
plt.subplot(121)
sns.histplot(data["ApplicantIncome"], kde=True)

plt.subplot(122)
data["ApplicantIncome"].plot.box(figsize=(16,5))
plt.show()
```

**[Output]**

Plaintext

```
<Figure size 1600x500 with 2 Axes>```
```
(Visualization/Plot)

---

**Code Block:**
```python
data.groupby("Loan_Status")['ApplicantIncome'].mean()
```

**[Output]**

Plaintext

```
Loan_Status
N    5446.078125
Y    5384.068720
Name: ApplicantIncome, dtype: float64```
```

---

**Code Block:**
```python
# HW: Try ttest here
from scipy.stats import ttest_ind
a = data[data["Loan_Status"]=="Y"]["ApplicantIncome"]
b = data[data["Loan_Status"]=="N"]["ApplicantIncome"]
ttest_ind(a, b)
# High p-value implies no relation between income and loan status
```

**[Output]**

Plaintext

TtestResult(statistic=np.float64(-0.11650844828724542), pvalue=np.float64(0.907287812130518), df=np.float64(612.0))```

---

**Code Block:**
```python
data.groupby("Loan_Status")['ApplicantIncome'].mean().plot.bar()
plt.ylabel("Mean Income of applicant")
plt.show()
```

**[Output]**

Plaintext

<Figure size 640x480 with 1 Axes>```
(Visualization/Plot)

---

## Bin income

**Code Block:**
```python
bins=[0,2500,4000,6000, 8000, 10000, 20000, 40000, 81000]
group=['Low','Average','medium', 'H1', 'h2', 'h3', 'h4' , 'Very high']
```

---

**Code Block:**

Python

```
data['Income_bin']= pd.cut(data['ApplicantIncome'],bins,labels=group) # Create new categorical column
```

---

**Code Block:**

Python

```
data.head()
```

**[Output]**

Plaintext

```
 Gender Married Dependents     Education Self_Employed  ApplicantIncome  \
0   Male      No          0      Graduate            No             5849
1   Male     Yes          1      Graduate            No             4583
2   Male     Yes          0      Graduate           Yes             3000
3   Male     Yes          0  Not Graduate            No             2583
4   Male      No          0      Graduate            No             6000

   CoapplicantIncome  LoanAmount  Loan_Amount_Term  Credit_History  \
0                0.0         NaN             360.0             1.0
1             1508.0       128.0             360.0             1.0
2                0.0        66.0             360.0             1.0
3             2358.0       120.0             360.0             1.0
4                0.0       141.0             360.0             1.0

  Property_Area Loan_Status Income_bin
0         Urban           Y     medium
1         Rural           N     medium
2         Urban           Y    Average
3         Urban           Y    Average
4         Urban           Y     medium  ```
```

(Table/Dataframe result)

---

**Code Block:**

```python
pd.crosstab(data["Income_bin"],data["Loan_Status"], normalize="index")
```

**[Output]**

Plaintext

```
Loan_Status         N         Y
Income_bin
Low          0.314815  0.685185
Average      0.296460  0.703540
medium       0.314685  0.685315
H1           0.370370  0.629630
h2           0.290323  0.709677
h3           0.325000  0.675000
h4           0.333333  0.666667
Very high    0.333333  0.666667```
```

(Table/Dataframe result)

---

**Code Block:**
```python
from scipy.stats import chi2_contingency
val = pd.crosstab(index=data["Income_bin"], columns=data["Loan_Status"]).values
print(val)
chi2_contingency(val) # chi_stat, p_value, df, expected_values
```

**[Output]**

Plaintext

```
[[ 34  74]
 [ 67 159]
 [ 45  98]
 [ 20  34]
 [  9  22]
 [ 13  27]
 [  3   6]
 [  1   2]]
```

Plaintext

```
Chi2ContingencyResult(statistic=np.float64(1.2420001711303135),
pvalue=np.float64(0.9899274842922701), dof=7, expected_freq=array([[ 33.77198697,
74.22801303],
       [ 70.67100977, 155.32899023],
       [ 44.71661238,  98.28338762],
       [ 16.88599349,  37.11400651],
       [  9.69381107,  21.30618893],
       [ 12.50814332,  27.49185668],
       [  2.81433225,   6.18566775],
       [  0.93811075,   2.06188925]]))
```

---

**Code Block:**
```python
Income_bin = pd.crosstab(data["Income_bin"],data["Loan_Status"], normalize="index")
Income_bin.plot(kind="bar",figsize=(4,4))
plt.xlabel("ApplicantIncome")
plt.ylabel("Percentage")
```

plt.show()

**[Output]**

Plaintext

<Figure size 400x400 with 1 Axes>```
(Visualization/Plot)

---

**Code Block:**
```python
data["TotalIncome"] = data["ApplicantIncome"] + data["CoapplicantIncome"]

---

**Code Block:**

Python

data.head()

**[Output]**

Plaintext

```
 Gender Married Dependents     Education Self_Employed  ApplicantIncome  \
0   Male      No          0      Graduate            No             5849
1   Male     Yes          1      Graduate            No             4583
2   Male     Yes          0      Graduate           Yes             3000
3   Male     Yes          0  Not Graduate            No             2583
4   Male      No          0      Graduate            No             6000

   CoapplicantIncome  LoanAmount  Loan_Amount_Term  Credit_History  \
0                0.0         NaN             360.0             1.0
1             1508.0       128.0             360.0             1.0
2                0.0        66.0             360.0             1.0
3             2358.0       120.0             360.0             1.0
4                0.0       141.0             360.0             1.0

  Property_Area Loan_Status Income_bin  TotalIncome
0         Urban           Y     medium       5849.0
1         Rural           N     medium       6091.0
2         Urban           Y    Average       3000.0
3         Urban           Y    Average       4941.0
```
```

4 Urban Y medium 6000.0 ```
(Table/Dataframe result)

---

**Code Block:**
```python
bins = [0,3000,5000,8000,81000]
group = ['Low','Average','High','Very High']
data["TotalIncome_bin"] = pd.cut(data["TotalIncome"],bins,labels=group)
```

---

**Code Block:**

Python

```
data.head()
```

**[Output]**

Plaintext

```
 Gender Married Dependents     Education Self_Employed  ApplicantIncome  \
0   Male      No          0      Graduate            No             5849
1   Male     Yes          1      Graduate            No             4583
2   Male     Yes          0      Graduate           Yes             3000
3   Male     Yes          0  Not Graduate            No             2583
4   Male      No          0      Graduate            No             6000

   CoapplicantIncome  LoanAmount  Loan_Amount_Term  Credit_History  \
0                0.0         NaN             360.0             1.0
1             1508.0       128.0             360.0             1.0
2                0.0        66.0             360.0             1.0
3             2358.0       120.0             360.0             1.0
4                0.0       141.0             360.0             1.0

  Property_Area Loan_Status Income_bin  TotalIncome TotalIncome_bin
0         Urban           Y     medium       5849.0            High
1         Rural           N     medium       6091.0            High
2         Urban           Y    Average       3000.0             Low
3         Urban           Y    Average       4941.0         Average
4         Urban           Y     medium       6000.0            High ```
```
(Table/Dataframe result)

---

**Code Block:**
```python
vals = pd.crosstab(data["TotalIncome_bin"], data["Loan_Status"])
vals
```

**[Output]**

Plaintext

```
Loan_Status       N    Y
TotalIncome_bin
Low              20   27
Average          69  154
High             61  151
Very High        42   90```
```
(Table/Dataframe result)

---

**Code Block:**
```python
vals_n = pd.crosstab(data["TotalIncome_bin"], data["Loan_Status"], normalize="index")
vals_n.plot(kind="bar",figsize=(4,4))
```

**[Output]**

Plaintext

```
<Axes: xlabel='TotalIncome_bin'>```
```text
<Figure size 400x400 with 1 Axes>```
```
(Visualization/Plot)

---

**Code Block:**
```python
chi2_contingency(vals) # chi_stat, p-value, dof, expec
# Stick to null hypothesis: no differnce of loan status across income bins
```

**[Output]**

Plaintext
```

Chi2ContingencyResult(statistic=np.float64(3.428480885250809), pvalue=np.float64(0.33015705640767123), dof=3, expected_freq=array([[ 14.6970684 ,  32.3029316 ],
       [ 69.73289902, 153.26710098],
       [ 66.29315961, 145.70684039],
       [ 41.27687296,  90.72312704]]))```

---

**Code Block:**
```python
data['Loan_Amount_Term'].value_counts()
```

**[Output]**

Plaintext

```
Loan_Amount_Term
360.0    512
180.0     44
480.0     15
300.0     13
240.0      4
84.0       4
120.0      3
60.0       2
36.0       2
12.0       1
Name: count, dtype: int64```
```

---

**Code Block:**
```python
data['Loan_Amount_Term'] = (data['Loan_Amount_Term']/12).astype('float')
```

---

**Code Block:**

Python

```
sns.countplot(x='Loan_Amount_Term', data=data)
plt.xlabel("Term in years")
plt.show()
```

**[Output]**

Plaintext

<Figure size 640x480 with 1 Axes>```
(Visualization/Plot)

---

**Code Block:**
```python
plt.figure(figsize=(16,5))
plt.subplot(121)
sns.histplot(data['LoanAmount']);

plt.subplot(122)
sns.boxplot(data=data, x='Loan_Status', y = 'LoanAmount')

plt.show()
```

**[Output]**

Plaintext

<Figure size 1600x500 with 2 Axes>```
(Visualization/Plot)

---

**Code Block:**
```python
data.head()
```

**[Output]**

Plaintext

```
 Gender Married Dependents     Education Self_Employed  ApplicantIncome  \
0   Male      No          0      Graduate            No             5849
1   Male     Yes          1      Graduate            No             4583
2   Male     Yes          0      Graduate           Yes             3000
3   Male     Yes          0  Not Graduate            No             2583
4   Male      No          0      Graduate            No             6000

   CoapplicantIncome  LoanAmount  Loan_Amount_Term  Credit_History  \
0                0.0         NaN              30.0             1.0
```

```
1          1508.0      128.0          30.0          1.0
2             0.0      66.0          30.0          1.0
3          2358.0      120.0          30.0          1.0
4             0.0      141.0          30.0          1.0

  Property_Area Loan_Status Income_bin  TotalIncome TotalIncome_bin
0         Urban           Y     medium       5849.0            High
1         Rural           N     medium       6091.0            High
2         Urban           Y    Average       3000.0             Low
3         Urban           Y    Average       4941.0         Average
4         Urban           Y     medium       6000.0            High  ```
```
(Table/Dataframe result)

---

**Code Block:**
```python
data.columns
```

**[Output]**

```Plaintext
Index(['Gender', 'Married', 'Dependents', 'Education', 'Self_Employed',
       'ApplicantIncome', 'CoapplicantIncome', 'LoanAmount',
       'Loan_Amount_Term', 'Credit_History', 'Property_Area', 'Loan_Status',
       'Income_bin', 'TotalIncome', 'TotalIncome_bin'],
      dtype='object')```
```

---

## Ability to pay EMI

**Code Block:**
```python
data['Loan_Amount_per_year']  = data['LoanAmount']/data['Loan_Amount_Term']
# Crude approximation without interest
```

---

**Code Block:**

```Python
data.head()
```

**[Output]**

Plaintext

```
 Gender Married Dependents     Education Self_Employed  ApplicantIncome  \
0   Male      No          0      Graduate            No             5849
1   Male     Yes          1      Graduate            No             4583
2   Male     Yes          0      Graduate           Yes             3000
3   Male     Yes          0  Not Graduate            No             2583
4   Male      No          0      Graduate            No             6000

   CoapplicantIncome  LoanAmount  Loan_Amount_Term  Credit_History  \
0                0.0         NaN              30.0             1.0
1             1508.0       128.0              30.0             1.0
2                0.0        66.0              30.0             1.0
3             2358.0       120.0              30.0             1.0
4                0.0       141.0              30.0             1.0

  Property_Area Loan_Status Income_bin  TotalIncome TotalIncome_bin  \
0         Urban           Y     medium       5849.0            High
1         Rural           N     medium       6091.0            High
2         Urban           Y    Average       3000.0             Low
3         Urban           Y    Average       4941.0         Average
4         Urban           Y     medium       6000.0            High

   Loan_Amount_per_year
0                   NaN
1              4.266667
2              2.200000
3              4.000000
4              4.700000  ```
```

(Table/Dataframe result)

---

**Code Block:**

```python
data['EMI'] = data['Loan_Amount_per_year']*1000/12 # Crude approximation without interest
data.head()
```

**[Output]**

Plaintext

```
 Gender Married Dependents     Education Self_Employed  ApplicantIncome  \
0   Male      No          0      Graduate            No             5849
```

```
1   Male     Yes          1      Graduate           No            4583
2   Male     Yes          0      Graduate          Yes            3000
3   Male     Yes          0  Not Graduate           No            2583
4   Male      No          0      Graduate           No            6000

   CoapplicantIncome  LoanAmount  Loan_Amount_Term  Credit_History  \
0                0.0         NaN              30.0             1.0
1             1508.0       128.0              30.0             1.0
2                0.0        66.0              30.0             1.0
3             2358.0       120.0              30.0             1.0
4                0.0       141.0              30.0             1.0

  Property_Area Loan_Status Income_bin  TotalIncome TotalIncome_bin  \
0         Urban           Y     medium       5849.0            High
1         Rural           N     medium       6091.0            High
2         Urban           Y    Average       3000.0             Low
3         Urban           Y    Average       4941.0         Average
4         Urban           Y     medium       6000.0            High

   Loan_Amount_per_year         EMI
0                   NaN         NaN
1              4.266667  355.555556
2              2.200000  183.333333
3              4.000000  333.333333
4              4.700000  391.666667  ```
(Table/Dataframe result)

---

**Code Block:**
```python
data['Able_to_pay_EMI'] = (data['TotalIncome']*0.2 > data['EMI']).astype('int')
```

---

**Code Block:**

```Python
data.head()
```

**[Output]**

```Plaintext
 Gender Married Dependents     Education Self_Employed  ApplicantIncome  \
```
```

```
0   Male      No          0      Graduate           No            5849
1   Male     Yes          1      Graduate           No            4583
2   Male     Yes          0      Graduate          Yes            3000
3   Male     Yes          0  Not Graduate           No            2583
4   Male      No          0      Graduate           No            6000

   CoapplicantIncome  LoanAmount  Loan_Amount_Term  Credit_History  \
0                0.0         NaN              30.0             1.0
1             1508.0       128.0              30.0             1.0
2                0.0        66.0              30.0             1.0
3             2358.0       120.0              30.0             1.0
4                0.0       141.0              30.0             1.0

  Property_Area Loan_Status Income_bin  TotalIncome TotalIncome_bin  \
0         Urban           Y     medium       5849.0            High
1         Rural           N     medium       6091.0            High
2         Urban           Y    Average       3000.0             Low
3         Urban           Y    Average       4941.0         Average
4         Urban           Y     medium       6000.0            High

   Loan_Amount_per_year         EMI  Able_to_pay_EMI
0                   NaN         NaN                0
1              4.266667  355.555556                1
2              2.200000  183.333333                1
3              4.000000  333.333333                1
4              4.700000  391.666667                1  ```
```
(Table/Dataframe result)

---

**Code Block:**
```python
sns.countplot(x='Able_to_pay_EMI', data = data, hue = 'Loan_Status')
```

**[Output]**

Plaintext

```
<Axes: xlabel='Able_to_pay_EMI', ylabel='count'>```
```text
<Figure size 640x480 with 1 Axes>```
```
(Visualization/Plot)

---

**Code Block:**
```

```python
vals = pd.crosstab(data['Able_to_pay_EMI'], data['Loan_Status'])
vals
```

**[Output]**

```Plaintext
Loan_Status        N    Y
Able_to_pay_EMI
0                 20   26
1                172  396
```

(Table/Dataframe result)

---

**Code Block:**

```python
chi2_contingency(pd.crosstab(data['Able_to_pay_EMI'], data['Loan_Status']))
# Low p-value implies there is a relation between "Able to pay EMI" feature and Loan status
```

**[Output]**

```Plaintext
Chi2ContingencyResult(statistic=np.float64(2.8614383727767136),
pvalue=np.float64(0.09072633762510059), dof=1, expected_freq=array([[ 14.38436482,
31.61563518],
       [177.61563518, 390.38436482]]))
```

---

# Dependents

**Code Block:**

```python
data['Dependents'].value_counts()
```

**[Output]**

```Plaintext
Dependents
0     345
1     102
```

2 101
3+ 51
Name: count, dtype: int64```

---

**Code Block:**
```python
data['Dependents'].replace('3+',3,inplace=True)
```

**[Output]**

Plaintext

```
/var/folders/j_/njpl090n1z9832p7my5n1s0c0000gn/T/ipykernel_8330/3782002166.py:1:
FutureWarning: A value is trying to be set on a copy of a DataFrame or Series through chained
assignment using an inplace method.
The behavior will change in pandas 3.0. This inplace method will never work because the
intermediate object on which we are setting values always behaves as a copy.

For example, when doing 'df[col].method(value, inplace=True)', try using 'df.method({col: value},
inplace=True)' or df[col] = df[col].method(value) instead, to perform the operation inplace on the
original object.


  data['Dependents'].replace('3+',3,inplace=True)
```

---

**Code Block:**

Python

```
data['Dependents'] = data['Dependents'].astype('float')
```

---

**Code Block:**

Python

```
sns.countplot(data =data, x = 'Dependents', hue = 'Loan_Status')
```

**[Output]**

Plaintext

<Axes: xlabel='Dependents', ylabel='count'>```
```text
<Figure size 640x480 with 1 Axes>```
(Visualization/Plot)

---

# Credit history

**Code Block:**
```python
data['Credit_History'].value_counts()
```

**[Output]**

Plaintext

Credit_History
1.0    475
0.0     89
Name: count, dtype: int64```

---

**Cod...
```python\n{source}\n```\n")

```
        outputs = cell.get('outputs', [])
        if outputs:
            output.append("**[Output]**")
            for out in outputs:
                if out.get('output_type') == 'stream':
                    text = "".join(out.get('text', []))
                    output.append(f"```text\n{text}```")
                elif out.get('output_type') in ['execute_result', 'display_data']:
                    data = out.get('data', {})
                    if 'text/plain' in data:
                        text = "".join(data['text/plain'])
                        output.append(f"```text\n{text}```")
                    if 'text/html' in data:
                        output.append("(Table/Dataframe result)")
                    if any(key.startswith('image/') for key in data.keys()):
                        output.append("(Visualization/Plot)")
            output.append("\n---\n")
```

```
        else:
            output.append("\n---\n")

    return "\n".join(output)

print(format_ipynb_for_word('fe_01.ipynb'))
```

**Code Block:**

```Python
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
```

---

Find some relation between features and loan status

**Code Block:**

```Python
data = pd.read_csv('loan.csv')
data.shape
```

**[Output]**

```Plaintext
(614, 13)
```

---

**Code Block:**

```Python
data.head()
```

**[Output]**

*(Table result showing columns like Loan_ID, Gender, Married, Dependents, Education, ApplicantIncome, etc.)*

---

**Code Block:**

```python
data.isna().sum()
```

**[Output]**

```plaintext
Gender              13
Married             3
Dependents          15
Education           0
Self_Employed       32
ApplicantIncome     0
CoapplicantIncome   0
LoanAmount          22
Loan_Amount_Term    14
Credit_History      50
Property_Area       0
Loan_Status         0
dtype: int64
```

---

# Univariate Analysis

**Code Block:**

```python
sns.countplot(data=data, x='Loan_Status')
plt.show()
```

**[Output]**

*(Visualization: Count plot of Loan_Status)*

---

**Code Block:**

```python
plt.subplot(121)
```

```
sns.histplot(data["ApplicantIncome"], kde=True)

plt.subplot(122)
data["ApplicantIncome"].plot.box(figsize=(16,5))
plt.show()
```

**[Output]**

*(Visualization: Histogram and Boxplot of ApplicantIncome)*

---

**Code Block:**

Python

```
# T-test for relation between ApplicantIncome and Loan_Status
from scipy.stats import ttest_ind
a = data[data["Loan_Status"]=="Y"]["ApplicantIncome"]
b = data[data["Loan_Status"]=="N"]["ApplicantIncome"]
ttest_ind(a, b)
# High p-value implies no significant relation
```

**[Output]**

Plaintext

```
TtestResult(statistic=-0.11650844828724542, pvalue=0.907287812130518, df=612.0)
```

---

**Code Block:**

Python

```
# Binning ApplicantIncome
bins=[0,2500,4000,6000, 8000, 10000, 20000, 40000, 81000]
group=['Low','Average','medium', 'H1', 'h2', 'h3', 'h4' , 'Very high']
data['Income_bin']= pd.cut(data['ApplicantIncome'], bins, labels=group)

from scipy.stats import chi2_contingency
val = pd.crosstab(index=data["Income_bin"], columns=data["Loan_Status"]).values
chi2_contingency(val)
```

**[Output]**

Plaintext

```
Chi2ContingencyResult(statistic=1.2420001711303135, pvalue=0.9899274842922701, dof=7, ...)
```

---

**Code Block:**

```Python
data["TotalIncome"] = data["ApplicantIncome"] + data["CoapplicantIncome"]
bins = [0,3000,5000,8000,81000]
group = ['Low','Average','High','Very High']
data["TotalIncome_bin"] = pd.cut(data["TotalIncome"], bins, labels=group)

vals = pd.crosstab(data["TotalIncome_bin"], data["Loan_Status"])
chi2_contingency(vals)
```

**[Output]**

```Plaintext
Chi2ContingencyResult(statistic=3.428480885250809, pvalue=0.33015705640767123, dof=3, ...)
```

---

**Code Block:**

```Python
# Ability to pay EMI feature
data['Loan_Amount_per_year'] = data['LoanAmount'] / (data['Loan_Amount_Term']/12)
data['EMI'] = data['Loan_Amount_per_year'] * 1000 / 12
data['Able_to_pay_EMI'] = (data['TotalIncome'] * 0.2 > data['EMI']).astype('int')

sns.countplot(x='Able_to_pay_EMI', data=data, hue='Loan_Status')
```

**[Output]**

*(Visualization: Count plot of EMI repayment ability vs Loan Status)*

---

**Code Block:**

```Python
chi2_contingency(pd.crosstab(data['Able_to_pay_EMI'], data['Loan_Status']))
```

**[Output]**

```Plaintext
Chi2ContingencyResult(statistic=2.8614383727767136, pvalue=0.09072633762510059, dof=1, ...)
```

---

# Credit history

**Code Block:**

```Python
data['Credit_History'].value_counts()
```

**[Output]**

```Plaintext
Credit_History
1.0    475
0.0     89
Name: count, dtype: int64
```

---

**Code Block:**

```Python
pd.crosstab(data['Credit_History'], data['Loan_Status'])
```

**[Output]**

```Plaintext
Loan_Status      N    Y
Credit_History
0.0             82    7
1.0             97  378
```

---

**Code Block:**

Python

```python
chi2_contingency(pd.crosstab(data['Credit_History'], data['Loan_Status']))
```

**[Output]**

Plaintext

```plaintext
Chi2ContingencyResult(statistic=174.63729658142535, pvalue=7.1847595487507425e-40, dof=1,
...)
```